Airport Lost Baggage and Lost Property Management System

Milestone 1 – Proposal and Architectural Requirements

Project name	Airport Lost Baggage and Lost Property Management System
Team name	Airport team
Milestone	M1 – Proposal and architectural requirements
Instructor / TA	Dr. Nada Mahmoud Hassan / Eng. Mohamed Fareh
Date	22nd of October 2026

Team members

#	Name	Student ID
1	Rana Yasser	231014860
2	Youssef Hussein	231004387
3	Yazan Loay	231014985
4	Yassin Wafik	231004431
5	Abdallah Mohamed	231004160

Contribution Table

Section	Member(s) responsible	Contribution
1. System overview	Yazan Loay	The overview and scope boundaries
2. Stakeholders and concerns	Yazan Loay	Stakeholder table
3. System context (C4 L1)	Yassin Wafik	Diagram and source file
4. Functional requirements	Youssef Hussein	User stories, MoSCoW, prototype scope
5. Quality attribute scenarios	Abdallah Mohamed	Six-part scenarios and measures
6. Constraints	Yazan Loay	Technical, business, organizational
7. ASRs and utility tree	Rana Yasser	Prioritization and justification
Review, formatting, references	Rana Yasser	Consistency check.

Project ideas Requirements

User Roles: Features three distinct roles: Passengers, Lost-Property Staff, and System Administrators.

Competing Quality Attributes: Balances strict transactional consistency (zero double handovers) and passenger data security against high usability and system availability during external service outages.

External Integration: Integrates with a third-party Email Delivery Service via API/SMTP to send asynchronous status updates.

Separated Architecture: Features a clearly separated browser frontend and modular backend REST API connected to a relational database.

Scope: Focuses purely on core lost-and-found operations at one airport, intentionally excluding live airline baggage scanners, security databases, and AI image recognition

1. System Overview

Problem: Airport lost-property handling can be difficult to track when reports and found items are recorded separately. Staff may need to compare descriptions manually, while passengers have limited information about the progress of a case. This can delay returns and make it harder to identify the correct owner.

Proposed System: The Airport Lost Baggage and Lost Property Management System is a web-based system for passengers and airport staff. Passengers can submit reports with details such as item category, color, description, last-seen location, date, and an optional photo. They can check case status and submit a claim when a possible match is identified. Lost-property staff register found items, review suggested matches, verify ownership, and record handovers. Administrators manage accounts, permissions, categories, and locations. The system can send email notifications when a case changes or a possible match is found.

Intended Audience: Passenger (reports items, tracks cases, claims items), Lost-property / baggage staff (registers found items, processes cases, verifies ownership, returns items), System administrator (manages accounts, permissions, item categories, and airport locations).

Case Lifecycle: Reported -> Under review -> Match suggested -> Claim pending -> Ownership verified -> Returned -> Closed. Every transition is recorded in a status history.

Architectural Significance: The system includes familiar data-entry and search functions, but its workflow creates important architectural decisions. A found item must not be handed to two different claimants, even if staff act at the same time. Passenger information must be protected without making reports difficult to submit. The system should continue handling cases if the email service is temporarily unavailable, while keeping hosting and maintenance costs low. It should also be possible to add categories or locations without changing many parts of the application. A separate frontend, a modular backend, a relational database, and an external email service are a practical starting point. The design can be refined in the next milestone.

2. Stakeholders and Concerns

Passengers - Travelers who lose or find items; some are non-technical, stressed, using mobile browsers.
Key Concerns: Easy and quick reporting; visibility of case status; privacy of personal data and photos; timely notification; not losing their item to someone else.

Lost-property staff - Airport baggage and lost-property officers handling daily cases.
Key Concerns: Fast registration of found items; useful match suggestions; clear case queue; protection against releasing an item wrongly; accountability for their actions.

System administrator - Manages accounts, roles, configuration and monitors the system.
Key Concerns: Simple user and permission management; configurable categories and locations; reliable data; accurate statistics.

Airport operations management - Owns the service and its reputation; funds it.
Key Concerns: Low operating cost; passenger satisfaction; fewer complaints and disputes; legal exposure; return-rate metrics.

Software development team - Builds, tests, and updates the application.
Key Concerns: Clear module boundaries, testability, maintainable code, and changes that do not require redesigning the whole system.

IT operations team - Deploys, monitors, and maintains the running service.
Key Concerns: Straightforward deployment, useful logs, backups, recovery procedures, and predictable operating costs.

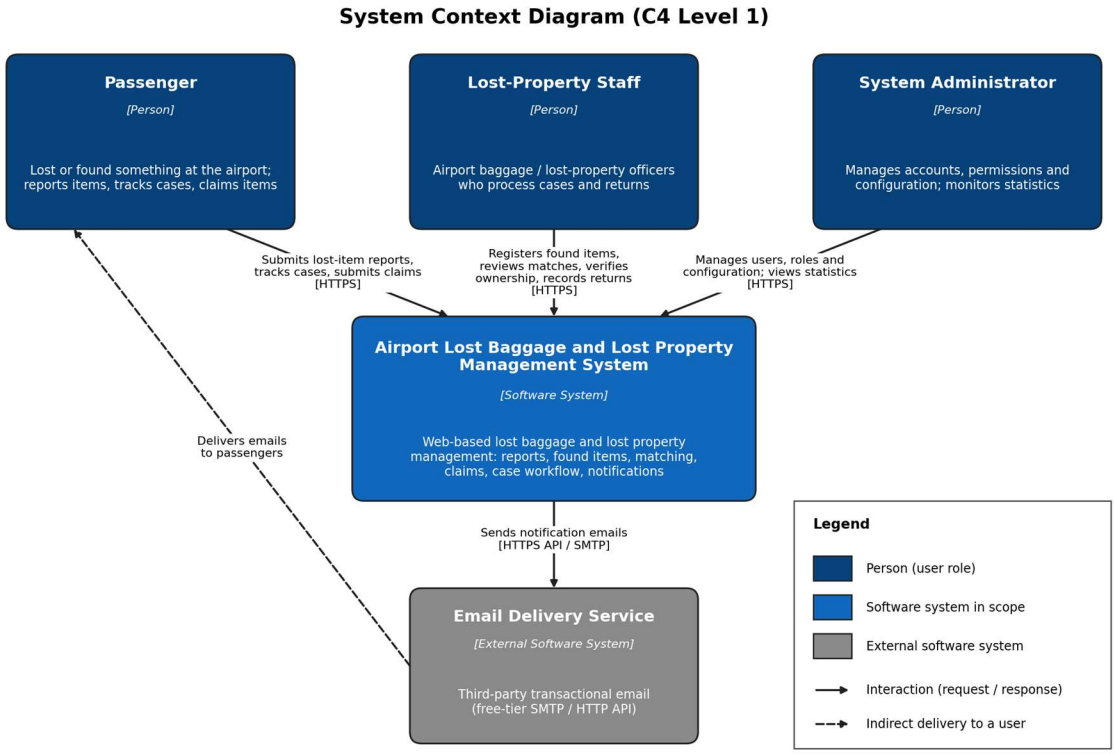
Regulators - Oversee personal-data handling.
Key Concerns: Lawful processing, data minimization, retention limits, access control, breach handling.

Airport IT and security team - Reviews the system's access controls and its fit with airport IT practices.
Key Concerns: Protection of passenger information, controlled access, traceable changes, and manageable integration with existing IT services.

3. System Context

The system operates as a single application bounded context, interacting with three distinct user roles and one external service. Case information and uploaded photos are managed securely within the system boundary.

- Passenger: Uses the passenger portal over HTTPS; receives email notifications.
- Lost-Property Staff: Uses the staff dashboard over HTTPS to process cases and returns.
- System Administrator: Uses the admin dashboard over HTTPS to manage configuration.
- Email Delivery Service (External): The system calls its API/SMTP to send asynchronous notifications.



4. Functional Requirements (MoSCoW & Scope)

ID	Requirement / Feature	Target User	Priority (MoSCoW)	MVP Scope
FR-01	Secure role-based registration and login	All Users	Must Have	Yes
FR-02	Submit lost-item reports with category, description, and optional photo	Passengers	Must Have	Yes
FR-03	View case progress and status history	Passengers	Must Have	Yes
FR-04	Register found items into the system inventory	Staff	Must Have	Yes
FR-05	System-generated ranked match suggestions between lost and found items	System / Staff	Must Have	Yes
FR-06	Review reports, assign cases, and update workflow statuses	Staff	Must Have	Yes
FR-07	Submit ownership claims and verify proof of ownership	Passengers / Staff	Must Have	Yes
FR-08	Record verified handovers and explicitly prevent double allocation	Staff	Must Have	Yes
FR-09	Automated email notifications on status changes and matches	Passengers	Should Have	Yes
FR-10	Basic management of user and staff permissions	Admins	Should Have	Yes
FR-11	Search and filter capabilities for inventory and case queues	Staff	Should Have	Yes
FR-12	Configuration of item categories and airport locations without code changes	Admins	Should Have	Yes
FR-13	Password reset via email link	All Users	Could Have	No (Post-MVP)
FR-14	Admin dashboard with business statistics (return rates, open cases)	Admins	Could Have	No (Post-MVP)
FR-15	Comprehensive audit logging of staff actions and automated retention rules	Admins	Could Have	No (Post-MVP)
FR-16	Integration with airline baggage-tracking, security systems, or AI image recognition	System	Won't Have	No (Out of Scope)

5. Quality Attribute Scenarios

QAS - Performance: Passenger submits a report with a 5 MB photo. At least 95% of submissions complete in under 3 seconds under simulated load (20 concurrent users).

QAS - Consistency: Two staff members simultaneously approve handover of the same item. Exactly one commits; the other receives a clear conflict error. Zero double allocations occur across 50 simulated attempts.

QAS - Security: A user modifies a URL to access another passenger's case. The request is denied (HTTP 403), logged, and zero data is leaked.

QAS - Availability (Backend): The backend process crashes. The service successfully restarts, and previously committed case data remains intact and available.

QAS - Availability (Email): The external email service goes down for 2 hours. Case updates continue to save normally. Notifications are stored and successfully retried after service recovery.

QAS - Modifiability: Admin adds a new item category via UI with zero code changes. Engineers add a new matching rule without impacting unrelated case-management code.

QAS - Usability: A stressed, first-time passenger reports an item on mobile. 90% of test users complete the report in under 4 minutes unaided (max 3 screens).

QAS - Testability: CI pipeline runs automated tests on matching and workflow modules using test doubles, verifying rules without sending real emails.

6. Architectural Constraints

- **Technical:** Cost-effective hosting utilizing a clear REST API over HTTPS between a decoupled frontend and backend. Email notifications require asynchronous queues to handle throttling. No integration with live external airline systems for the prototype.
- **Business:** Must operate within a limited ongoing budget. Strict compliance with data protection laws (e.g., Egyptian Law No. 151 of 2020) for personal data minimization. Ownership verification strictly requires human-in-the-loop authorization; no automated handovers.
- **Organizational:** Initial release targeted before the upcoming peak travel season. Strict IT governance requiring version-controlled source code, traceable architecture decisions, and stable API contracts across parallel engineering tracks.

7. Architecturally Significant Requirements (ASRs)

7.1 Prioritized Table

Requirements from Sections 4-6 are rated by business value (BV) and architectural impact (AI) as High (H), Medium (M), or Low (L). The top five are the primary ASRs.

Rank	ASR Focus	Refinement	Source	BV	AI
1	Consistency	No double allocation of an item	QAS-2, FR-08, C-B4	H	H
2	Security	Data isolation, role-based access, privacy compliance	QAS-3, FR-01, C-B2	H	H
3	Cost / deployability	Low-cost deployment	C-T1, C-B1	H	H
4	Availability	Survive email-provider outage	QAS-5, FR-09, C-T3	M	H
5	Modifiability	New category, location, matching rule	QAS-6, FR-12	M	H
6	Availability	Automatic recovery from backend crash	QAS-4	H	M
7	Usability	Quick first-time reporting	QAS-7, FR-02	H	M
8	Functionality (matching)	Ranked match suggestions	FR-05	H	M
9	Performance	Report submission latency	QAS-1	M	M
10	Testability	Isolated automated tests	QAS-8	M	M

7.2 Justification of the Top Five ASRs

ASR-1 Consistency (QAS-2): A found item must not be handed to more than one claimant. This requirement affects the database and the case workflow: the system needs to check the item's current state when a handover is confirmed and reject a conflicting second attempt. It is important to design this rule into the workflow rather than add it after the main features are built.

ASR-2 Security (QAS-3): The system stores passenger contact details and information about lost belongings. Authentication and role- and record-level authorization must therefore be enforced by the backend, not only by hiding pages in the interface. These checks influence the API and data access design. They also need to be balanced with usability so that passengers can report an item without unnecessary steps.

ASR-3 Cost and deployability (C-T1, C-B1): The airport operator has a limited operating budget, so the system should avoid unnecessary infrastructure and maintenance overhead. A separate frontend and one modular backend are more practical than a collection of independently deployed services. This keeps deployment simpler, although it limits the options for redundancy and high availability.

ASR-4 Resilience to email outages (QAS-5): Sending an email should not determine whether a case update is saved. The notification process should be separated from the main case workflow, with failed messages recorded for retry when the email service becomes available again. A simple database-backed queue is a reasonable starting point; a separate message-broker platform is not required for the initial version.

ASR-5 Modifiability (QAS-6): Different airports may use different item categories and location names. Keeping categories and locations as configuration data makes these changes possible without changing application code. Matching rules should also be kept in a clearly separated part of the backend so that they can be adjusted without affecting unrelated case-management functions. The design should remain simple enough to maintain.